# NPU

From FriendlyELEC WiKi

查看中文

## Contents


- 1 Introduction
- 2 Supported Platforms
- 3 The operating system used in this article
- 4 Check NPU driver version
- 5 RKNN Quick Example
  - 5.1 Installing RKNN Runtime
  - 5.2 Check rknn version
  - 5.3 C++ Example: Build and run the YOLOv5 demo
  - 5.4 Python Example: Install and Test RKNN Toolkit Lite2 on Debian11
    - 5.4.1 Install RKNN Toolkit Lite2
    - 5.4.2 Run the python example
  - 5.5 Python Example: Install and Test RKNN Toolkit Lite2 on Ubuntu 20.04 (Focal)
    - 5.5.1 Compile and install python3.9
    - 5.5.2 Install RKNN Toolkit Lite2
    - 5.5.3 Run the python example
- 6 RKLLM and Large Language Models
  - 6.1 Currently Supported Models
  - 6.2 Performance Comparison of Models
  - 6.3 Model Conversion (PC Side)
    - 6.3.1 Installing RKLLM-Toolkit
    - 6.3.2 Download and Convert the Model
  - 6.4 Compile and run demo application (On Development Board)
  - 6.5 Useful Resources
- 7 Doc
- 8 Other
  - 8.1 View NPU Load
  - 8.2 Set NPU freq
  - 8.3 View NPU freq


# 1 Introduction

This document will explain how to use the RKNN SDK, as well as how to convert large language models in Huggingface format into RKLLM format, and deploy them on RK3576/RK3588 for hardware-accelerated inference using the NPU.

# 2 Supported Platforms

- RK3566
- RK3568
- RK3576
- RK3588

# 3 The operating system used in this article

- debian-bullseye-desktop-arm64
- ubuntu-focal-desktop-arm64

Other versions of Debian/Ubuntu systems also support NPU at the driver and runtime library level, but if you want to develop using Python, you need to install the required Python version from the official SDK yourself.

# 4 Check NPU driver version

```
$ sudo cat /sys/kernel/debug/rknpu/version
RKNPU driver: v0.9.8
```

# 5 RKNN Quick Example

## 5.1 Installing RKNN Runtime

```
cd ~
export GIT_SSL_NO_VERIFY=1
git clone https://github.com/airockchip/rknn-toolkit2.git
cd rknn-toolkit2/rknpu2
sudo cp ./runtime/Linux/librknn_api/aarch64/* /usr/lib
sudo cp ./runtime/Linux/rknn_server/aarch64/usr/bin/* /usr/bin/
sudo cp ./runtime/Linux/librknn_api/include/* /usr/include/
```

## 5.2 Check rknn version

```
$ strings /usr/bin/rknn_server |grep 'build@'
2.3.0 (e80ac5c build@2024-11-07T12:52:53)
rknn_server version: 2.3.0 (e80ac5c build@2024-11-07T12:52:53)
$ strings /usr/lib/librknnrt.so |grep 'librknnrt version:'
librknnrt version: 2.3.0 (c949ad889d@2024-11-07T11:35:33)
```

## 5.3 C++ Example: Build and run the YOLOv5 demo

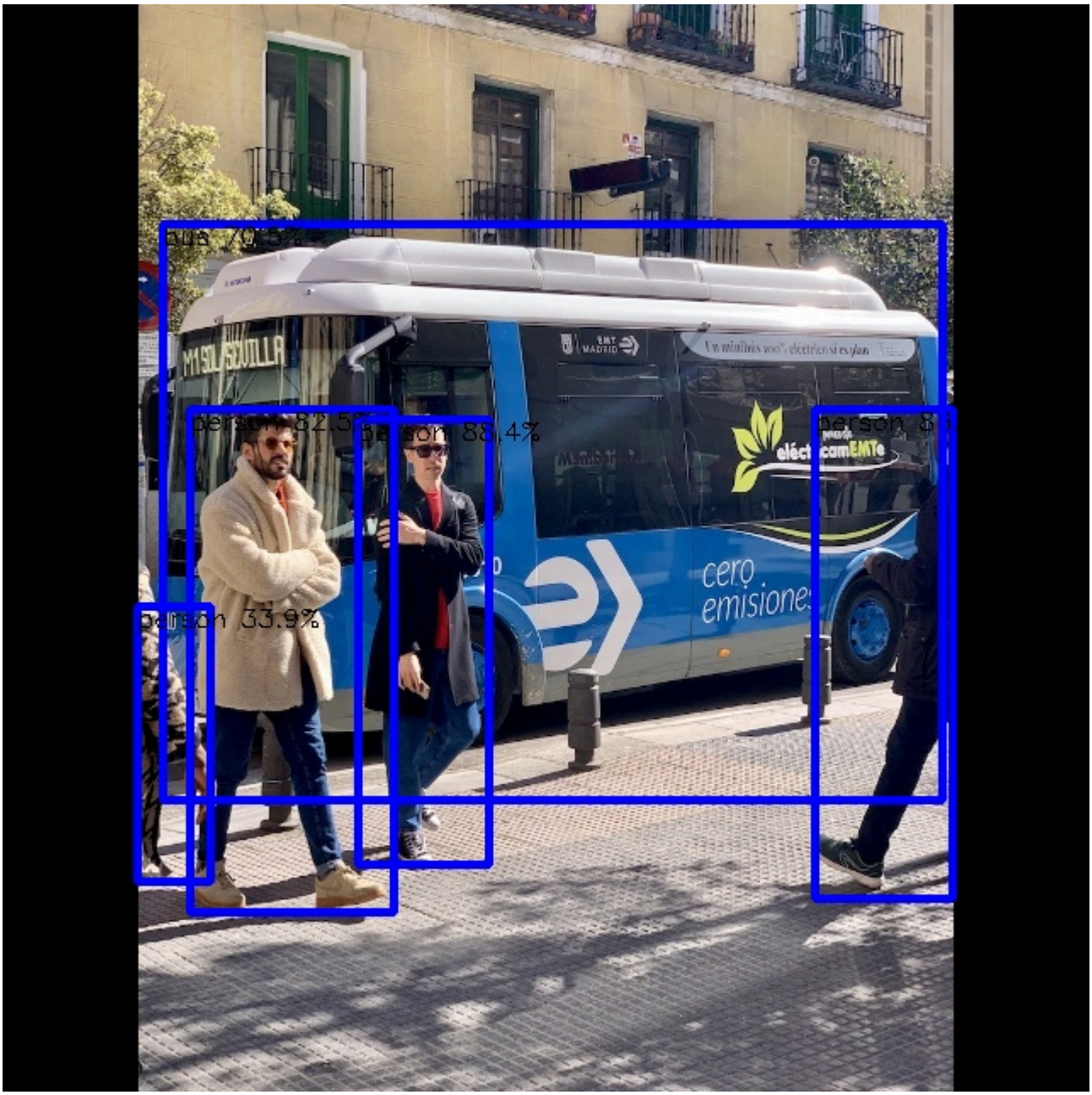
```
sudo apt-get update
sudo apt-get install -y gcc g++ make cmake

# fix broken link
cd ~/rknn-toolkit2/rknpu2/examples/3rdparty/mpp/Linux/aarch64
rm -f librockchip_mpp.so librockchip_mpp.so.1
ln -s librockchip_mpp.so.0 librockchip_mpp.so
ln -s librockchip_mpp.so.0 librockchip_mpp.so.1

cd ~/rknn-toolkit2/rknpu2/examples/rknn_yolov5_demo
chmod +x ./build-linux.sh
sudo ln -s /usr/bin/gcc /usr/bin/aarch64-gcc
sudo ln -s /usr/bin/g++ /usr/bin/aarch64-g++
export GCC_COMPILER=aarch64
./build-linux.sh -t rk3588 -a aarch64 -b Release
cd install/rknn_yolov5_demo_Linux
./rknn_yolov5_demo model/RK3588/yolov5s-640-640.rknn model/bus.jpg
```

Transfer the generated out.jpg to PC to view the result:

```
scp out.jpg xxx@YourIP:/tmp/
```




## 5.4 Python Example: Install and Test RKNN Toolkit Lite2 on Debian11

### 5.4.1 Install RKNN Toolkit Lite2

```
sudo apt-get update
sudo apt-get install -y python3-dev python3-numpy python3-opencv python3-pip
cd ~/rknn-toolkit2
pip3 install ./rknn-toolkit-lite2/packages/rknn_toolkit_lite2-2.3.0-cp39-cp39-manylinux_2_17_aarch64.manylinux20
```

### 5.4.2 Run the python example

```
$ cd ~/rknn-toolkit2/rknn-toolkit-lite2/examples/resnet18/
$ python3 test.py
W rknn-toolkit-lite2 version: 2.3.0
--> Load RKNN model
done
--> Init runtime environment
I RKNN: [06:29:43.602] RKNN Runtime Information, librknnrt version: 2.3.0 (c949ad889d@2024-11-07T11:35:33)
I RKNN: [06:29:43.602] RKNN Driver Information, version: 0.9.8
I RKNN: [06:29:43.602] RKNN Model Information, version: 6, toolkit version: 2.3.0(compiler version: 2.3.0 (c949a
W RKNN: [06:29:43.617] query RKNN_QUERY_INPUT_DYNAMIC_RANGE error, rknn model is static shape type, please expor
W Query dynamic range failed. Ret code: RKNN_ERR_MODEL_INVALID. (If it is a static shape RKNN model, please igno
```

```
done
--> Running model
resnet18
-----TOP 5-----
[812] score:0.999680 class:"space shuttle"
[404] score:0.000249 class:"airliner"
[657] score:0.000013 class:"missile"
[466] score:0.000009 class:"bullet train, bullet"
[895] score:0.000008 class:"warplane, military plane"
```

## 5.5 Python Example: Install and Test RKNN Toolkit Lite2 on Ubuntu 20.04 (Focal)

### 5.5.1 Compile and install python3.9

```
sudo apt install build-essential libssl-dev libffi-dev software-properties-common \
    libbz2-dev libncurses-dev libncursesw5-dev libgdbm-dev liblzma-dev libsqlite3-dev \
    tk-dev libgdbm-compat-dev libreadline-dev

wget https://www.python.org/ftp/python/3.9.21/Python-3.9.21.tar.xz
tar -xvf Python-3.9.21.tar.xz
cd Python-3.9.21/
./configure --enable-optimizations
make -j$(nproc)
sudo make install
```

### 5.5.2 Install RKNN Toolkit Lite2

```
python3.9 -m pip install --upgrade pip
python3.9 -m pip install opencv-python

cd ~
[ -d rknn-toolkit2 ] || git clone https://github.com/airockchip/rknn-toolkit2.git \
    --depth 1 -b master
cd rknn-toolkit2
python3.9 -m pip install \
    ./rknn-toolkit-lite2/packages/rknn_toolkit_lite2-2.3.0-cp39-cp39-manylinux_2_17_aarch64.manylinux2014_aarch6
    -i https://pypi.tuna.tsinghua.edu.cn/simple/
```

### 5.5.3 Run the python example

```
cd ~/rknn-toolkit2/rknn-toolkit-lite2/examples/resnet18/
python3.9 test.py
```

output:

```
W rknn-toolkit-lite2 version: 2.3.0
--> Load RKNN model
done
--> Init runtime environment
I RKNN: [07:08:29.756] RKNN Runtime Information, librknnrt version: 2.3.0 (c949ad889d@2024-11-07T11:35:33)
I RKNN: [07:08:29.757] RKNN Driver Information, version: 0.9.8
I RKNN: [07:08:29.762] RKNN Model Information, version: 6, toolkit version: 2.3.0(compiler version: 2.3.0 (c949a
W RKNN: [07:08:29.814] query RKNN_QUERY_INPUT_DYNAMIC_RANGE error, rknn model is static shape type, please expor
W Query dynamic range failed. Ret code: RKNN_ERR_MODEL_INVALID. (If it is a static shape RKNN model, please igno
done
--> Running model
resnet18
-----TOP 5-----
[812] score:0.999680 class:"space shuttle"
[404] score:0.000249 class:"airliner"
[657] score:0.000013 class:"missile"
[466] score:0.000009 class:"bullet train, bullet"
[895] score:0.000008 class:"warplane, military plane"
```

# 6 RKLLM and Large Language Models

## 6.1 Currently Supported Models

- InternLM2-1.8B (https://huggingface.co/internlm/internlm2-1_8b)
- TinyLLAMA-1.1B (https://huggingface.co/TinyLlama/TinyLlama-1.1B-Chat-v1.0)
- Qwen2.5-1.5B (https://huggingface.co/Qwen/Qwen2.5-1.5B)
- Qwen2.5-3B (https://huggingface.co/Qwen/Qwen2.5-3B)
- DeepSeek-R1-Distill-Qwen-1.5B (https://huggingface.co/deepseek-ai/DeepSeek-R1-Distill-Qwen-1.5B)
- DeepSeek-R1-Distill-Qwen-7B (https://huggingface.co/deepseek-ai/DeepSeek-R1-Distill-Qwen-7B)
- MiniCPM3-4B (https://huggingface.co/openbmb/MiniCPM3-4B)
- Llama-3.2-1B-Instruct (https://www.modelscope.cn/models/LLM-Research/Llama-3.2-1B-Instruct)
- Llama-3.1-8B (https://www.modelscope.cn/models/LLM-Research/Meta-Llama-3.1-8B)
- Gemma-2B (https://www.huggingface.co/google/gemma-2b)
- Gemma-2-9B-IT (https://www.huggingface.co/google/gemma-2-9b-it)
- ChatGLM3-6B (https://huggingface.co/THUDM/chatglm3-6b)

The RKLLM format files for the above models can be found in the following path on the cloud drive: “09_Other files/RKLLM models”. The cloud drive link is http://dl.friendlyelec.com/rk3576. Please refer to the official Rockchip link for model support: https://github.com/airockchip/rknn-llm

## 6.2 Performance Comparison of Models

| Hardware Model | AI Model | Parameter size | Dtype | Performance | CPU load | NPU load | Memory usage |
|---|---|---|---|---|---|---|---|
| NanoPi-M5 | TinyLLAMA | 1.1B | w4a16 | Pre filled: 166.61 token/s Reasoning: 17.90 token/s | 24% | 2*73% | 0.58G |
| NanoPi-M5 | DeepSeek-R1-Distill-Qwen | 1.5B | w4a16 | Pre filled: 111.84 token/s Reasoning: 11.75 token/s | 23% | 2*75% | 1.01G |
| NanoPi-M5 | DeepSeek-R1-Distill-Qwen | 7B | w4a16 | Pre filled: 38.05 token/s Reasoning: 4.12 token/s | 18.625% | 2*85% | 3.76G |
| NanoPi-M5 | Qwen2.5 | 1.5B | w4a16 | Pre filled: 109.36 token/s Reasoning: 11.23 token/s | 20.25% | 2*75% | 1.01G |
| NanoPi-M5 | Qwen2.5 | 3B | w4a16 | Pre filled: 70.06 token/s Reasoning: 8.17 token/s | 20.875% | 2*76% | 1.78G |
| NanoPi-M5 | InternLM2 | 1.8B | w4a16 | Pre filled: 96.08 token/s Reasoning: 11.72 token/s | 16.75% | 2*79% | 1.02G |
| NanoPi-M5 | MiniCPM3 | 4B | w4a16 | Pre filled: 39.29 token/s Reasoning: 4.11 token/s | 25.875% | 2*66% | 2.34G |
| NanoPi-M5 | Llama-3.2-Instruct | 1B | w4a16 | Pre filled: 180.33 token/s Reasoning: 14.51 token/s | 17.125% | 2*79% | 0.87G |
| NanoPi-M5 | Llama-3.1-8B | 8B | w4a16 | Pre filled: 36.44 token/s Reasoning: 3.8 token/s | 18.875% | 2*85% | 3.97G |
| NanoPi-M5 | Gemma-2 | 2B | w4a16 | Pre filled: 93.08 token/s Reasoning: 7.86 token/s | 19.875% | 2*83% | 1.65G |
| NanoPi-M5 | Gemma-2 | 9B | w4a16 | Pre filled: 25.91 token/s Reasoning: 2.84/s | 19.125% | 2*80% | 5.25G |
| NanoPi-M5 | ChatGLM3 | 6B | w4a16 | Pre filled: 47.13 token/s Reasoning: 4.03 token/s | 19.75% | 2*88% | 3.04G |

## 6.3 Model Conversion (PC Side)

The Hugging Face-format large language model needs to be converted into an RKLLM model to enable loading and usage on the Rockchip NPU platform. The conversion process must be performed on a PC, with Ubuntu 20.04 as the recommended operating system. A Docker container can also be used.

### 6.3.1 Installing RKLLM-Toolkit

```
sudo apt-get update
sudo apt-get install -y python3-dev python3-numpy python3-opencv python3-pip
cd ~
git clone https://github.com/airockchip/rknn-llm
cd rknn-llm
git reset 8623edd0559a07e7127876d685f2b7ca8b83590c --hard
pip3 install rkllm-toolkit/packages/rkllm_toolkit-1.1.4-cp38-cp38-linux_x86_64.whl
```

### 6.3.2 Download and Convert the Model

Use the git clone command to download the required model from the Hugging Face repository. Update test.py to point to the downloaded model, then execute test.py to start the model conversion process:

```
cd ~/rknn-llm/rkllm-toolkit/examples/
git lfs install
git clone https://huggingface.co/deepseek-ai/DeepSeek-R1-Distill-Qwen-1.5B
sed -i 's|^modelpath\s*=.*|modelpath = "./DeepSeek-R1-Distill-Qwen-1.5B"|' test.py
sed -i 's|^dataset\s*=.*|dataset = None|' test.py
# If the CPU is not RK3588, such as RK3576, it also needs to be modified as follows:
sed -i "s/target_platform='rk3588'/target_platform='rk3576'/g" test.py
sed -i "s/num_npu_core=3/num_npu_core=2/g" test.py
```

Start converting the model by executing test.py:

```
python3 test.py
```

The output information of the model conversion is as follows:

```
ubuntu@20de568f4011:~/rknn-llm/rkllm-toolkit/examples$ python3 test.py
INFO: rkllm-toolkit version: 1.1.4
The argument `trust_remote_code` is to be used with Auto classes. It has no effect here and is ignored.
Optimizing model: 100%|[redacted]| 2
Building model: 100%|[redacted]| 399/
WARNING: The bos token has two ids: 151646 and 151643, please ensure that the bos token ids in config.json and t
INFO: The token_id of bos is set to 151646
INFO: The token_id of eos is set to 151643
INFO: The token_id of pad is set to 151643
Converting model: 100%|[redacted]| 339/339
INFO: Exporting the model, please wait ....
[==================================================>] 597/597 (100%)
INFO: Model has been saved to ./qwen.rkllm!
```

If successful, the file qwen.rkllm will be generated. Rename it to DeepSeek-R1-Distill-Qwen-1.5B.rkllm and upload it to the development board:

```
mv qwen.rkllm DeepSeek-R1-Distill-Qwen-1.5B.rkllm
scp DeepSeek-R1-Distill-Qwen-1.5B.rkllm root@NanoPC-T6:/home/pi
```

## 6.4 Compile and run demo application (On Development Board)

```
sudo apt-get update
sudo apt-get install -y gcc g++ make cmake

# get rknn-llm
cd ~
[ -d rknn-llm ] || git clone https://github.com/airockchip/rknn-llm
cd rknn-llm
git reset 8623edd0559a07e7127876d685f2b7ca8b83590c --hard
cd examples/rkllm_api_demo
chmod +x ./build-linux.sh
sed -i 's/^GCC_COMPILER_PATH=.*/GCC_COMPILER_PATH=aarch64-linux-gnu/' ./build-linux.sh
./build-linux.sh

# run llm_demo
cd ./build/build_linux_aarch64_Release/
export LD_LIBRARY_PATH=~/rknn-llm/rkllm-runtime/Linux/librkllm_api/aarch64:$LD_LIBRARY_PATH
ulimit -HSn 10240
# max_new_tokens = 512, max_context_len = 1024
taskset f0 ./llm_demo ~/DeepSeek-R1-Distill-Qwen-1.5B.rkllm 512 1024
```

Output:

```
rkllm init start
I rkllm: rkllm-runtime version: 1.1.4, rknpu driver version: 0.9.8, platform: RK3588

rkllm init success

**********************可输入以下问题对应序号获取回答/或自定义输入********************

[0] 把下面的现代文翻译成文言文: 到了春风和煦，阳光明媚的时候，湖面平静，没有惊涛骇浪，天色湖光相连，一片碧绿，广阔无际；沙洲上的鸥鸟，
[1] 以咏梅为题目，帮我写一首古诗，要求包含梅花、白雪等元素。
[2] 上联: 江边惯看千帆过
[3] 把这句话翻译成中文: Knowledge can be acquired from many sources. These include books, teachers and practical ex
[4] 把这句话翻译成英文: RK3588是新一代高端处理器，具有高算力、低功耗、超强多媒体、丰富数据接口等特点

*************************************************************************


user: 天空为什么是蓝色的?
robot: 这个问题看起来很有趣，但我不太确定。我想知道天空的颜色是什么原因导致的，以及是否有其他因素影响了它的颜色。
首先，我想到的是太阳本身的颜色。太阳看起来像一个红色的东西，对吧？那是不是和天空的颜色有关呢？
然后，我还记得以前学过一点关于大气层的知识，说大气层是透明的，但有一定的折射作用。这可能影响了我们看到的天空颜色。
另外，我还听说过光的反射和折射的一些定律，比如费马原理，可能也与天空的颜色有关。
不过，我还不太确定这些因素是如何相互作用的，以及它们最终导致了天空为什么会是蓝色的。

也许我可以先从太阳本身的颜色开始分析。太阳看起来红色，是因为它发出的光大部分在红光区，而蓝光和绿光被大气层吸收或散射，所以我们看到的是
另外，我还记得有人说过，天空的颜色可能与地球上的天气状况有关，比如阳光是否直射、云的类型等等。
不过，这些信息我还不太确定，需要进一步查证。

总的来说，我觉得天空的颜色是由多种因素共同作用的结果，包括太阳的颜色、大气层的折射和反射、光的反射定律以及天气状况等。但具体来说，为什
</think>

天空呈现蓝色的原因主要归因于光的折射和反射现象，以及地球表面光线分布的影响。以下是详细的解释：

1. **太阳的颜色**：太阳本身看起来红色，这是因为太阳发出的大部分光在红光区，而蓝光和绿光被大气层吸收或散射，所以我们看到的是红色。

2. **大气层的作用**：
   - 大气层是透明的，但具有折射作用。当阳光穿过大气层时，部分光线会被折射，导致天空呈现蓝色。
   - 这种现象称为“大气折射”或“海市蜃楼效应”。不同颜色的光在不同介质中的折射率不同，导致它们在空气中传播时发生偏折。

3. **光的反射和折射**：
   - 光线在空气中的传播遵循费马原理，即光线会尽可能地快速到达目标。这包括反射和折射。
   - 太阳光穿过大气层后，在地球表面反射，而部分光被折射进入天空，

user:
```

## 6.5 Useful Resources

- https://github.com/airockchip/rknn-llm
- https://github.com/wudingjian/rkllm_chat

# 7 Doc

https://github.com/rockchip-linux/rknpu2/tree/master/doc

# 8 Other

## 8.1 View NPU Load

```
sudo cat /sys/kernel/debug/rknpu/load
```

## 8.2 Set NPU freq

```
echo userspace | sudo tee /sys/class/devfreq/fdab0000.npu/governor > /dev/null
echo 800000000 | sudo tee /sys/class/devfreq/fdab0000.npu/min_freq > /dev/null
echo 1000000000 | sudo tee /sys/class/devfreq/fdab0000.npu/max_freq > /dev/null
```

## 8.3 View NPU freq

```
cat /sys/class/devfreq/fdab0000.npu/cur_freq
```

Retrieved from "https://wiki.friendlyelec.com/wiki/index.php?title=NPU&oldid=27474"

- This page was last modified on 15 September 2025, at 06:25.